Frontend Developer — Take-Home Assignment

Candidate Instructions

Welcome

Thank you for taking the time to work on this assignment. It's designed to give you a realistic slice of the kind of work you'd do on the job, and to give us a fair, practical way to evaluate your experience, technical skills, and creativity. There's no single "correct" solution — we're more interested in how you think, structure your code, and make decisions.

Please read through all three parts before you start, and reach out if anything is unclear — we'd rather answer a question upfront than have you guess.

Assignment Overview

This assignment has three parts. Please complete them in order, as later parts build on earlier ones.

- Part 1: API Documentation & Feature Implementation — document the given API in your own way, then build a chat application screen using it.
- Part 2: Creative Landing Page — design and build a landing page to showcase what you built in Part 1.
- Part 3: Thought Process Write-up — a short summary of your approach, decisions, and anything you ran into along the way.

Part 1 — API Documentation & Feature Implementation

You'll be given a set of API endpoints to build a chat application feature.

What to do

- Start by writing your own API documentation for the given endpoint(s) — endpoints, methods, request/response structure, parameters, etc. Use any format you're comfortable with (Markdown, a Postman collection, OpenAPI/Swagger, etc.). Do this first, before you start building. This is a standalone deliverable — it shows us how you'd design/interpret a clean API structure.
  - You're free to rename endpoints/routes if you'd design them better, and to add or remove endpoints as you see fit — it's entirely your call.
- Then, using the given mock data/API directly, build the following:
  - Login — a login page where the user enters a phone number and sets their name to log in. There's no separate registration flow — if the phone number is new/unique, the API registers it as a new user automatically.
  - Starting a conversation — to start a new conversation, the user searches by a number or name, then starts the conversation.
  - Group conversations — support creating a group conversation with multiple participants, in addition to one-to-one conversations.
  - Message list — the full conversation history, with sender and receiver clearly distinguished visually, and each message timestamped.
  - Sending messages — users can send a new message; empty messages should not be sendable.

  - Real-time updates — new incoming messages should appear automatically, without the user needing to refresh.
  - Loading, empty, and error states — handled appropriately throughout.
  - Auto-scroll — the view should auto-scroll to the latest message by default, but should not force-scroll the user down if they've scrolled up to read earlier messages.
- Write clean, maintainable, and reasonably organized code — treat this as production code, not a throwaway prototype.
- Deploy this to a live, hosted URL (e.g., Vercel, Netlify) — this link is required at submission.

Where to focus: all of the above matters, but if you're deciding where to spend the most care and polish, make it the chat panel itself (the message list, sending, and real-time behavior) — that's the core of the experience and where we'll be looking closest.

Bonus: candidates who take it one step further — adding a thoughtful extra element that shows original, one-step-ahead thinking (a smart edge-case handled gracefully, a small interaction that improves the experience, a detail that isn't asked for but adds real value) — can earn extra credit for it.

This bonus only applies if the addition is genuinely original. A common or generic addition won't count toward it, even if it's well executed.

Part 2 — Creative Landing Page

Design and build a landing page that presents/showcases the feature you built in Part 1, as if you were introducing it to real users. No design file will be provided for this part — the visual direction is entirely up to you.

What to do

- Design the layout, color palette, typography, and any animation/interaction yourself.
- Make sure the page is responsive and clearly communicates what the feature does.
- Feel free to be bold here — we'd rather see your own creative instincts than a generic template.
- Deploy this page to a live, hosted URL as well — this link is required at submission.

Bonus: candidates who take it one step further — adding a thoughtful extra element that shows original, one-step-ahead thinking (an unexpected interaction, a clever addition to the flow, a detail that isn't asked for but adds real value) — can earn extra credit for it.

This bonus only applies if the addition is genuinely original. A common or generic addition — a stock testimonial section, a standard FAQ accordion, and the like — won't count toward it, even if it's well executed.

Part 3 — Thought Process Write-up

Once you've completed Parts 1–2, write a brief summary covering your approach. Include it in your README or as a separate document.

Please cover

- Why you chose your architecture/libraries/approach in Part 1, and any trade-offs you considered.
- The reasoning behind your design choices in Part 2.

- How you used AI tools (if at all) — which tool(s), what you used them for (e.g., boilerplate, debugging, drafting the API documentation, research), and what you changed, rejected, or wrote yourself instead of relying on the AI's output.
- What you'd improve or do differently with more time.

Any Issues You Ran Into

While working with the given API, if you noticed anything odd, inconsistent, or broken — unexpected response shapes, missing/incorrect error handling, odd status codes, pagination quirks, or anything else — describe it here: what you noticed, and how you handled or worked around it in your implementation. If you didn't run into anything, that's fine too — just say so.

> *Tip: Keep this concise and honest. We're not looking for a perfect answer — we're looking for clear reasoning.*

Assignment Logistics

Time to complete	24 hours
Tech stack	React / Next.js
API docs (Swagger)	https://frontend-task-chatapp.onrender.com/docs/
Submission deadline	Aug 22, 2026 4:00 PM

How to Submit

- Push your complete code to a GitHub repository (public, or private with access granted to us).
- Include a README with setup/run instructions, the tech stack you used, and your Part 3 write-up.
- A live, hosted demo link is required for both Part 1 (the implemented screens) and Part 2 (the landing page) — e.g., Vercel, Netlify, or similar. Submissions without working demo links will not be reviewed.
- Once ready, send us the repository link and both demo links at [submission email/link].

A Few Notes

- You're welcome to use AI tools, libraries, or any resources you'd normally use on the job — just document how you used them in your Part 3 write-up (see above).
- If you get stuck or something in the assignment seems ambiguous, make a reasonable assumption, note it in your write-up, and keep moving.
- We value clear, working code over a large number of features — a smaller, well-built solution is better than a rushed, incomplete one.

Good luck — we're looking forward to seeing what you build!